Technical guide to hardening AI-generated code and secure use of GenAI in the SDLC

This annex expands in depth on the **Technical Mitigation Guide** presented in section 4.4, turning it into a standalone, actionable reference document. Its content derives directly from the systemic vulnerability patterns identified during the empirical audit of code generated by ChatGPT, Claude and Gemini, and translates each finding into concrete technical countermeasures, contrasted code examples (vulnerable version versus hardened version) and process controls that can be integrated into the software development lifecycle.

How to use this guide

The guide is organized into two complementary parts. **Part I** brings together hardening strategies oriented to the artifact, that is, to fixing the specific weaknesses of the code before its deployment. **Part II** provides process recommendations for incorporating generative Artificial Intelligence into development without introducing risk. The Part I sheets can be consulted independently as a review checklist; the Part II recommendations are intended to be institutionalized at the team level.

The code examples are expressed in Python with the Flask microframework, as it was the environment used in the study, but the principles can be extrapolated to any technology stack. The color **red** identifies the vulnerable pattern and the color **green** the hardened implementation.

Part I. Hardening strategies for generated code

The audit showed that the three models produce functional but architecturally vulnerable code, with critical severities (CVSS ≥ 9.0) in eight of the nine modules evaluated. Effective hardening does not depend on a single control, but on the overlapping of several defensive layers: it is precisely this combination that allowed Gemini to reduce the maximum severity of its file upload module from Critical to Medium. The following model illustrates this defense-in-depth logic, which the subsequent sheets implement layer by layer.

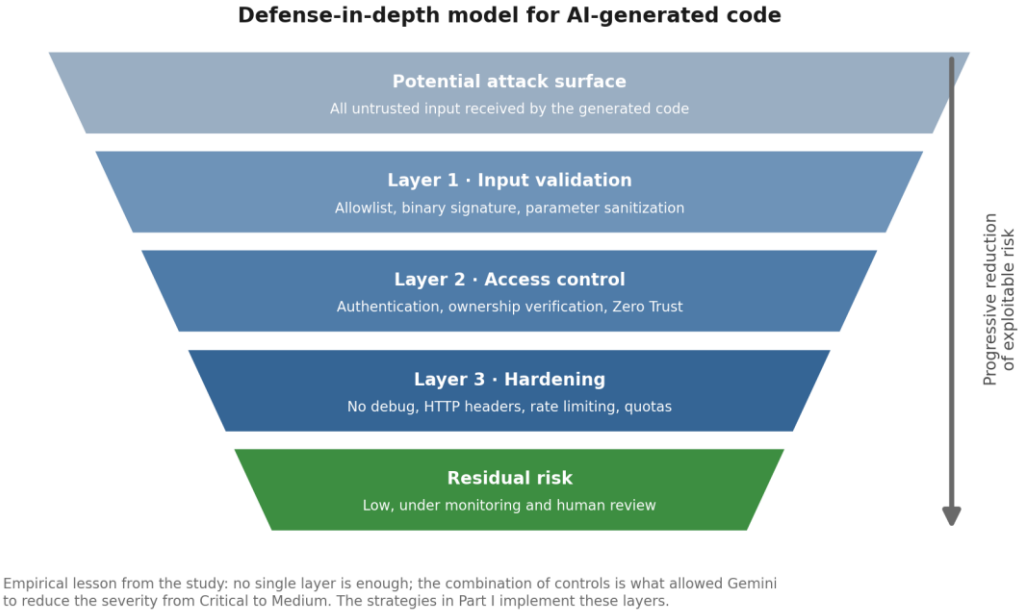


Figure A1. *Defense-in-depth model for AI-generated code* (Source: Author's own work.)

Before detailing each countermeasure, the following matrix summarizes which category of the OWASP Top 10:2025 standard each hardening control covers, so that the joint coverage of the set of measures can be verified.

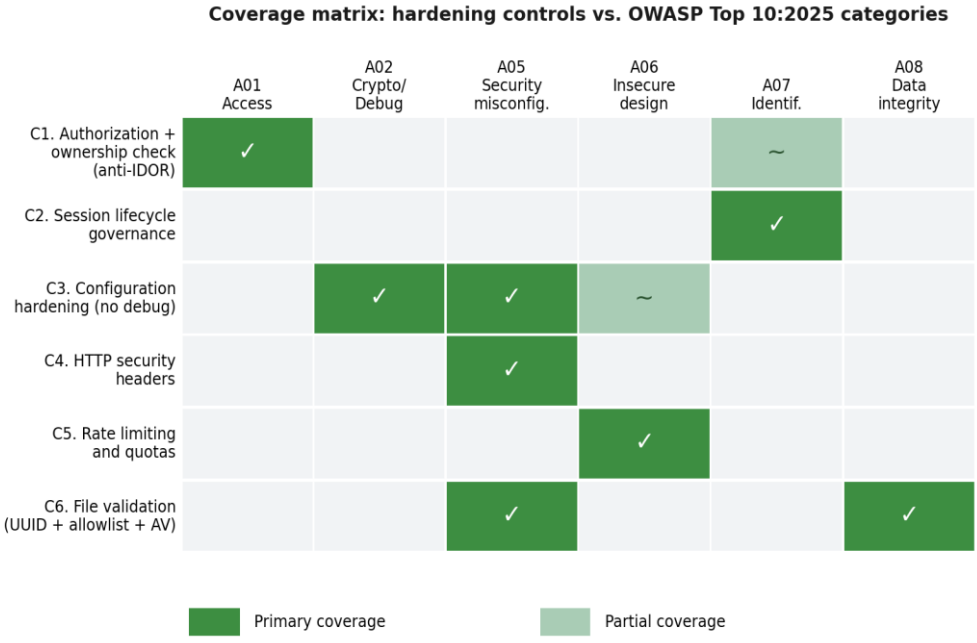


Figure A2. *Coverage matrix of the hardening controls against the OWASP categories* (Source: Author's own work.)

C1. Broken access control and insecure direct object references (A01:2025)

This was the most universal flaw in the study. The models coded the business logic under an assumption of implicit trust: ChatGPT and Gemini exposed destructive methods (PUT and DELETE) without requiring a session token, allowing anonymous alteration of records, while ChatGPT and Claude did not protect direct object references (IDOR), so that an attacker could access other users' records by iterating the numeric identifier in the URL. The countermeasure combines two inseparable checks: requiring a valid session and, in addition, verifying that the authenticated user owns the requested resource.

OWASP category	A01:2025 — Broken Access Control
Weakness (CWE)	CWE-862 (Missing authorization), CWE-639 (IDOR)
Evidence in the study	ChatGPT and Gemini CRUD API (CVSS up to 9.3); IDOR in ChatGPT and Claude
Control objective	Apply Zero Trust: per-request authorization + ownership verification

Vulnerable code

```
@app.route("/api/records/<int:rec_id>", methods=["GET", "DELETE"])
def record(rec_id):
    # No session and no ownership check:
    # anyone can read or delete any record (IDOR + broken access)
    rec = Record.query.get(rec_id)
    if request.method == "DELETE":
        db.session.delete(rec); db.session.commit()
    return jsonify(rec.to_dict())
```

Hardened code

```
@app.route("/api/records/<int:rec_id>", methods=["GET", "DELETE"])
@login_required                       # 1) requires a valid session
def record(rec_id):
    rec = Record.query.get_or_404(rec_id)
    if rec.owner_id != current_user.id:   # 2) verifies ownership (anti-IDOR)
        abort(403)
    if request.method == "DELETE":
        db.session.delete(rec); db.session.commit()
        return "", 204
    return jsonify(rec.to_dict())
```

Checklist

- ☐ All endpoints, and especially PUT/PATCH/DELETE, require authentication.
- ☐ Every operation on a resource checks ownership or role before being executed.
- ☐ Identifiers exposed in the URL do not allow enumeration of other users' resources.

C2. Governance of the session and identity lifecycle (A07:2025)

Identity management was universally deficient. Claude issued reusable recovery tokens (CVSS 9.1) and did not revoke active sessions after a credential change (CWE-613 / CWE-640), and cookies were generally generated without the relevant security attributes. Hardening requires single-use tokens with an expiration, global session invalidation upon any sensitive change, and cookies flagged as secure.

OWASP category	A07:2025 — Identification and Authentication Failures
Weakness (CWE)	CWE-613 (Insufficient expiration), CWE-640 (Weak recovery)
Evidence in the study	Claude authentication: reusable tokens (9.1) and unrevoked sessions (7.6)
Control objective	Ephemeral single-use tokens, global revocation and hardened cookies

Vulnerable code

```
# Recovery token always valid and reusable
def make_reset_token(user):
    return hashlib.md5(str(user.id).encode()).hexdigest()  # predictable, no expiration

# Session cookie without protection
app.config["SESSION_COOKIE_SECURE"]   = False
app.config["SESSION_COOKIE_HTTPONLY"] = False
```

Hardened code

```
from itsdangerous import URLSafeTimedSerializer
s = URLSafeTimedSerializer(app.secret_key)

def make_reset_token(user):
    return s.dumps({"uid": user.id, "v": user.session_version}, salt="pwd-reset")

def consume_reset_token(token):
    data = s.loads(token, salt="pwd-reset", max_age=900)   # expires after 15 min
    user = User.query.get(data["uid"])
    user.session_version += 1     # invalidates ALL sessions and the token itself
    return user

app.config.update(SESSION_COOKIE_SECURE=True,
                  SESSION_COOKIE_HTTPONLY=True,
                  SESSION_COOKIE_SAMESITE="Lax")
```

Checklist

- ☐ Recovery tokens expire and can only be used once.
- ☐ A password or role change revokes all of the user's active sessions.
- ☐ Session cookies carry the Secure, HttpOnly and SameSite attributes.

C3. Hardening of the deployment configuration (A02:2025 / A05:2025)

The models prioritized developer convenience over environment hardening. The absolute maximum score of the study (CVSS 10.0, Claude CRUD API) stemmed from leaving the debug=True directive active, which exposes an interactive console capable of executing arbitrary code; Claude also introduced a /setup endpoint that acted as an unauthenticated backdoor. Hardening consists of separating the configuration by environment, disabling debugging in production, replacing error pages with versions that do not leak stack traces, and removing development endpoints.

OWASP category	A05:2025 — Security Misconfiguration (and A02 in debug RCE)
Weakness (CWE)	CWE-489 (Active debug code), CWE-215 (Debug information leak)
Evidence in the study	debug=True in Claude (CVSS 10.0); unauthenticated /setup endpoint (9.3)
Control objective	No debugging in production, opaque errors and minimal attack surface

Vulnerable code

```
@app.route("/setup")          # backdoor: initializes the DB without authentication
def setup():
    db.create_all(); return "DB ready"

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)   # exposed interactive console -> RCE
```

Hardened code

```
import os
app.config["DEBUG"] = os.environ.get("FLASK_ENV") == "development"

@app.errorhandler(500)
def error_500(e):
    return render_template("500.html"), 500   # does not expose the stack trace

# The /setup endpoint is removed; initialization is a deployment task
# (migration) run outside the exposed application.

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=False)
```

Checklist

- ☐ debug=True is disabled in any environment accessible from the network.
- ☐ Error pages do not reveal traces, versions or internal paths.
- ☐ No configuration, initialization or test endpoints exist in production.
- ☐ Server version banners (Werkzeug/Python) are hidden.

C4. HTTP security headers (A05:2025)

During the dynamic analysis phase, it was found that all three generators omitted the fundamental security headers, leaving the interfaces exposed to Clickjacking and Cross-Site Scripting. Declaring them centrally at a single exit point guarantees that every response includes them.

OWASP category	A05:2025 — Security Misconfiguration
Weakness (CWE)	CWE-1021 (Frame restriction), CWE-693 (Protection mechanism failure)

Evidence in the study	Absence of CSP and X-Frame-Options in all three models (DAST)
Control objective	Apply defensive headers to all responses

Hardened code

```
@app.after_request
def set_secure_headers(resp):
    resp.headers["Content-Security-Policy"]   = "default-src 'self'"
    resp.headers["X-Frame-Options"]           = "DENY"          # anti-Clickjacking
    resp.headers["X-Content-Type-Options"]    = "nosniff"
    resp.headers["Strict-Transport-Security"] = "max-age=63072000; includeSubDomains"
    resp.headers["Referrer-Policy"]           = "no-referrer"
    return resp
```

Checklist

- ☐ Every response includes CSP, X-Frame-Options, X-Content-Type-Options and HSTS.
- ☐ The CSP policy restricts origins to those strictly necessary.

C5. Rate limiting and resource control (A06:2025)

No model anticipated abuse of the infrastructure: the AIs allowed unlimited login attempts — facilitating brute force— and omitted transactional limits, enabling the resource exhaustion observed in Claude (disk quota), ChatGPT (memory) and Gemini (request flooding). The countermeasure introduces rate limiting at sensitive points and global consumption caps.

OWASP category	A06:2025 — Vulnerable Components and Insecure Design
Weakness (CWE)	CWE-307 (Unlimited attempts), CWE-770 (Resources without quota)
Evidence in the study	Login without limit; disk/memory/request exhaustion in all three models
Control objective	Stop brute force and prevent denial of service

Hardened code

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
limiter = Limiter(get_remote_address, app=app, default_limits=["200 per hour"])

@app.route("/login", methods=["POST"])
@limiter.limit("5 per minute")          # stops dictionary brute force
def login():
    ...

app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024   # upload cap (5 MB)
```

Checklist

- ☐ Authentication endpoints apply rate limiting or progressive lockout.
- ☐ Per-user quotas and global limits on storage and request size are in place.

C6. Robust file upload validation (CWE-434 / A08:2025)

This scenario offers the most valuable lesson of the study. While ChatGPT and Claude allowed the unrestricted upload of dangerous files (potential RCE, CVSS 9.4), Gemini reduced the severity to Medium (5.3) by applying two patterns: strict allowlist validation and decoupling of the file's identity through a UUID, which neutralized null byte injection and directory traversal. Nevertheless, not even Gemini inspected the content for malware, letting the EICAR test file through (CWE-494). Complete hardening combines the three controls.

OWASP category	A05:2025 (upload) and A08:2025 (integrity / antivirus)
Weakness (CWE)	CWE-434 (Unrestricted upload), CWE-494 (No integrity verification)
Evidence in the study	RCE in ChatGPT/Claude (9.4); Gemini contained it to 5.3 but did not detect EICAR
Control objective	Content-based allowlist + UUID renaming + antivirus inspection

Vulnerable code

```
f = request.files["file"]
# Trusts the name and extension declared by the client
f.save(os.path.join(UPLOAD_DIR, f.filename))   # allows malware.php, photo.jpg.exe...
```

Hardened code

```
import uuid, magic   # python-magic: detects the type by its actual binary signature
ALLOWED = {"image/jpeg": ".jpg", "image/png": ".png", "application/pdf": ".pdf"}

f = request.files["file"]
mime = magic.from_buffer(f.read(2048), mime=True)   # does NOT trust the header
f.seek(0)
if mime not in ALLOWED:                             # content-based allowlist
    abort(400)
safe_name = f"{uuid.uuid4().hex}{ALLOWED[mime]}"    # decouples identity (UUID)
dest = os.path.join(UPLOAD_DIR, safe_name)          # folder outside the webroot
f.save(dest)
if not antivirus_scan(dest):                        # content inspection (EICAR)
    os.remove(dest); abort(422)
```

Checklist

- ☐ The file type is validated by its binary signature, not by the extension or the header.
- ☐ Files are renamed with an unguessable identifier and stored outside the webroot.
- ☐ The content undergoes antivirus inspection before becoming available.

Part II. Recommendations for secure use of GenAI in the SDLC

Fixing individual flaws is necessary but insufficient. The secure integration of GenAI requires process controls distributed throughout the development lifecycle, under a central premise derived from the study: generative Artificial Intelligence must be treated as a coding accelerator and never as a substitute for human security judgment. The following figure places each recommendation in the SDLC phase where it provides the most value.

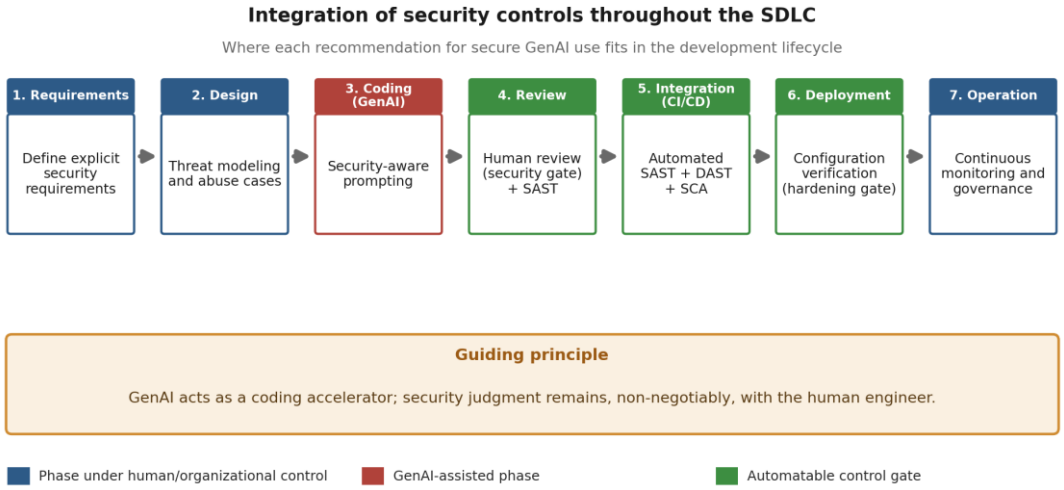


Figure A3. *Integration of the secure GenAI use controls throughout the SDLC* (Source: Author's own work.)

R1. Security-aware prompting

Left to their default configuration, the models solve only the "happy path" and omit hardening. Explicitly including the non-functional security requirements in the instruction itself substantially changes the quality of the generated code.

Weak prompt (induces insecure code)

```
Create a Flask REST API to manage users with CRUD operations.
```

Security-aware prompt

```
Create a Flask REST API to manage users with CRUD operations.
Mandatory security requirements:
  1. Token authentication on all endpoints and verification of
     resource ownership to prevent IDOR.
  2. Validation and sanitization of all inputs.
  3. Rate limiting on authentication endpoints.
  4. HTTP security headers (CSP, X-Frame-Options, HSTS).
  5. No debug=True and no configuration or initialization endpoints.
  6. Error handling that does not expose traces or versions.
Briefly justify each security control you apply.
```

R2. Mandatory human review as a security gate

No piece of AI-generated code should reach production without an expert security review. The study shows that even functionally correct code is architecturally vulnerable and that AI transfers the responsibility for hardening entirely to the engineer.

Actions

- Treat generated code as a proposal, not as a finished solution.

- Prioritize the review of authorization, session and file-handling flows, which concentrated the critical severities.
- Record the security review in version control (reviewer, findings, fixes).

R3. Automation of SAST, DAST and SCA in continuous integration

The empirical evidence justifies incorporating automated analysis specifically aimed at AI-generated code, closing the loop between the speed that GenAI provides and the assurances that professional development demands. A quality gate in the pipeline stops the promotion of the code if findings appear above the accepted threshold.

Example of a security gate in CI/CD (excerpt)

```
security-gate:
  stage: test
  script:
    - semgrep --config=auto --error           # SAST
    - pip-audit                               # SCA: vulnerable dependencies
    - zap-baseline.py -t "$STAGING_URL"       # DAST against the test environment
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

R4. Deployment configuration verification (hardening gate)

Insecure configurations were a pattern common to all three models. Before each release, the environment configuration must be systematically audited.

Actions

- Confirm that debugging and any development endpoints are disabled.
- Verify the absence of default credentials and the correct management of secrets.
- Check that the security headers and resource limits are active.

R5. Threat modeling and abuse case analysis

To compensate for the absence of a Zero Trust mental model in LLMs, it is advisable to deliberately force consideration of the edge cases and abuse vectors that the model ignores: session persistence and revocation, data ownership and consumption limits. Explicitly asking the question "how would an attacker abuse this functionality?" during design reveals omissions before coding.

R6. Software composition analysis (SCA)

AI may propose outdated libraries or libraries with known vulnerabilities. Incorporating dependency scanning into the supply chain —and keeping an up-to-date inventory of components— avoids carrying third-party risk into the final product.

R7. Governance, training and a culture of critical review

Secure adoption rests on a team-level GenAI usage policy, supported by secure coding checklists and a definition of "done" that includes security criteria. Training should guide developers to act as critical reviewers of generated code and not as passive recipients of it.

Reusable secure prompt template

The following template can be prepended to any code generation request to impose a security baseline consistent with the findings of this work:

```
Act as a security-focused software engineer. Generate <COMPONENT>
in <LANGUAGE/FRAMEWORK> while mandatorily complying with:
  - Zero Trust: authentication and ownership-based authorization on every operation.
  - Validation and sanitization of all untrusted input.
  - Secure session management: ephemeral tokens, revocation and hardened cookies.
  - Hardening: no debug mode, security headers, opaque errors.
  - Resilience: rate limiting and consumption quotas.
  - For file uploads: content-based allowlist, renaming and antivirus.
State explicitly which controls you applied and which security assumptions you make.
```

Definition-of-"done" checklist with security criteria

No increment that incorporates AI-generated code should be considered done without meeting the following criteria:

- ☐ Authorization and ownership verification present on all endpoints.
- ☐ Secure session lifecycle (ephemeral tokens, revocation, hardened cookies).
- ☐ Hardened configuration: no debugging, no development endpoints, managed secrets.
- ☐ HTTP security headers applied to all responses.
- ☐ Rate limiting and resource quotas configured.
- ☐ File upload validated by content, with renaming and antivirus inspection.
- ☐ SAST, DAST and SCA analyses passed with no findings above the threshold.
- ☐ Human security review performed and documented.

Taken together, this guide operationalizes the central conclusion of the work: GenAI is a productivity accelerator that must operate within a defense-in-depth framework in which security judgment remains, non-negotiably, in the hands of the engineer. The choice of Gemini as the recommended assistant was based precisely on the fact that it applied —albeit in a fragmentary way— several of these layers; systematizing them as described here makes it possible to bring any AI-generated code closer to a manageable level of residual risk.